

Relatório de Otimização com EXPLAIN ANALYZE

Resumo executivo

Este relatório documenta o ciclo de análise e otimização das oito consultas analíticas obrigatórias do projeto OficinaData, executadas em PostgreSQL 17.10 sobre o schema `oficina`. A metodologia adotada consistiu em coletar planos `EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)` para cada consulta antes e depois da introdução de índices candidatos, repetindo cada execução três vezes e considerando o menor tempo observado como medida representativa.

O dataset utilizado contém aproximadamente 22 mil registros distribuídos entre as tabelas operacionais (cliente: 200, veículo: 500, funcionário: 50, tipo_servico: 20, peça: 40, agendamento: 3.500, item_servico: 7.000, item_peca: 5.000, pagamento: 3.045, avaliação: 2.200, usuário: 6). Esse volume cabe integralmente em poucas centenas de páginas de 8 KB, o que se reflete diretamente nas decisões do otimizador.

O achado central é que, para este volume, o planejador do PostgreSQL seleciona corretamente `Hash Join` combinado a `Seq Scan` em todas as consultas com junções (Q3, Q4, Q5, Q8). Construir tabelas de hash sobre dimensões pequenas (20, 50 ou 200 linhas) é mais barato do que percorrer índices, e os dados inteiros residem em `shared_buffers` (todos os planos exibem `Buffers: shared hit`, sem `read`), o que anula o benefício teórico de índices que economizariam I/O.

Dos oito índices testados, apenas `idx_agendamento_veiculo` foi mantido em `db/05_indices.sql`, por ter sido efetivamente selecionado pelo otimizador para substituir `agendamento_pkey` no `Index Only Scan` da contagem em Q1, com redução de aproximadamente 5%. Os outros sete índices testados foram descartados porque o planejador continuou preferindo `Seq Scan/Hash Join`, o que é empiricamente correto para este dataset.

1. Metodologia

- Comando padrão de coleta: `EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)`.
- Antes da primeira coleta, executou-se `ANALYZE`; para garantir estatísticas atualizadas.
- Cada uma das oito consultas foi executada três vezes consecutivas; o tempo registrado é o mínimo das três execuções (atenua ruído de cache frio e variações de SO).
- Ambiente: PostgreSQL 17.10 sobre Windows 11; schema padrão `oficina`; cliente `psql`.
- Saídas brutas em `docs/.before_plans.txt` (sem índices criados) e `docs/.after_plans.txt` (com a bateria de índices candidatos).
- Volumes carregados: cliente 200, veículo 500, funcionário 50, tipo_servico 20, peça 40, agendamento 3.500 (2.450 Concluído / 350 Em andamento / 350 Agendado / 175 Cancelado / 175 No-show), item_servico 7.000, item_peca 5.000, pagamento 3.045, avaliação 2.200, usuário 6.

Resumo dos tempos mínimos observados:

Consulta	BEFORE (ms)	AFTER (ms)	Variação
Q1	1.872	1.772	-5.3%
Q2	1.343	1.318	-1.9%
Q3	2.644	2.740	+3.6% (ruído)
Q4	5.238	5.277	+0.7% (equivalente)
Q5	2.849	3.042	+6.8% (ruído)
Q6	1.216	1.213	-0.2% (equivalente)

Q7	0.019	0.019	0% (sem mudança)
Q8	4.184	4.637	+10.8% (ruído)

2. Análise por consulta

Consulta 1: contagem de registros por tabela

SQL executado

```
SELECT 'cliente' AS tabela, COUNT(*) AS qtd FROM cliente UNION ALL SELECT 'veiculo', COUNT(*) FROM veiculo UNION ALL SELECT 'funcionario', COUNT(*) FROM funcionario UNION ALL SELECT 'tipo_servico', COUNT(*) FROM tipo_servico UNION ALL SELECT 'peca', COUNT(*) FROM peca UNION ALL SELECT 'agendamento', COUNT(*) FROM agendamento UNION ALL SELECT 'item_servico', COUNT(*) FROM item_servico UNION ALL SELECT 'item_peca', COUNT(*) FROM item_peca UNION ALL SELECT 'pagamento', COUNT(*) FROM pagamento UNION ALL SELECT 'avaliacao', COUNT(*) FROM avaliacao;
```

Plano SEM índices (melhor de 3 execuções, 1.872 ms)

```
Append (cost=5.50..540.37 rows=10 width=40) (actual time=0.026..1.838 rows=10 loops=1) Buffers: shared hit=184
-> Aggregate (cost=5.50..5.51 rows=1 width=40) (actual time=0.026..0.026 rows=1 loops=1) Buffers: shared hit=3
-> Seq Scan on cliente (cost=0.00..5.00 rows=200 width=0) (actual time=0.008..0.017 rows=200 loops=1) Buffers:
shared hit=3 -> Aggregate (cost=11.25..11.26 rows=1 width=40) (actual time=0.046..0.046 rows=1 loops=1)
Buffers: shared hit=5 -> Seq Scan on veiculo (cost=0.00..10.00 rows=500 width=0) (actual time=0.004..0.029
rows=500 loops=1) Buffers: shared hit=5 -> Aggregate (cost=1.62..1.64 rows=1 width=40) (actual
time=0.008..0.008 rows=1 loops=1) Buffers: shared hit=1 -> Seq Scan on funcionario (cost=0.00..1.50 rows=50
width=0) (actual time=0.004..0.005 rows=50 loops=1) Buffers: shared hit=1 -> Aggregate (cost=1.25..1.26 rows=1
width=40) (actual time=0.005..0.005 rows=1 loops=1) Buffers: shared hit=1 -> Seq Scan on tipo_servico
(cost=0.00..1.20 rows=20 width=0) (actual time=0.003..0.004 rows=20 loops=1) Buffers: shared hit=1 ->
Aggregate (cost=1.50..1.51 rows=1 width=40) (actual time=0.006..0.007 rows=1 loops=1) Buffers: shared hit=1 ->
Seq Scan on peca (cost=0.00..1.40 rows=40 width=0) (actual time=0.003..0.005 rows=40 loops=1) Buffers: shared
hit=1 -> Aggregate (cost=153.53..153.54 rows=1 width=40) (actual time=0.360..0.360 rows=1 loops=1) Buffers:
shared hit=23 -> Index Only Scan using agendamento_pkey on agendamento (cost=0.28..144.78 rows=3500 width=0)
(actual time=0.006..0.237 rows=3500 loops=1) Heap Fetches: 0 Buffers: shared hit=23 -> Aggregate
(cost=146.50..146.51 rows=1 width=40) (actual time=0.552..0.553 rows=1 loops=1) Buffers: shared hit=59 -> Seq
Scan on item_servico (cost=0.00..129.00 rows=7000 width=0) (actual time=0.005..0.298 rows=7000 loops=1)
Buffers: shared hit=59 -> Aggregate (cost=101.50..101.51 rows=1 width=40) (actual time=0.400..0.400 rows=1
loops=1) Buffers: shared hit=39 -> Seq Scan on item_peca (cost=0.00..89.00 rows=5000 width=0) (actual
time=0.003..0.218 rows=5000 loops=1) Buffers: shared hit=39 -> Aggregate (cost=65.06..65.07 rows=1 width=40)
(actual time=0.251..0.251 rows=1 loops=1) Buffers: shared hit=27 -> Seq Scan on pagamento (cost=0.00..57.45
rows=3045 width=0) (actual time=0.004..0.143 rows=3045 loops=1) Buffers: shared hit=27 -> Aggregate
(cost=52.50..52.51 rows=1 width=40) (actual time=0.178..0.178 rows=1 loops=1) Buffers: shared hit=25 -> Seq
Scan on avaliacao (cost=0.00..47.00 rows=2200 width=0) (actual time=0.010..0.103 rows=2200 loops=1) Buffers:
shared hit=25 Planning Time: 0.197 ms Execution Time: 1.872 ms (45 linhas)
```

Diagnóstico. A consulta é um Append de 10 agregações independentes. Cada ramo é Seq Scan + Aggregate, exceto o ramo de agendamento, que originalmente usa Index Only Scan using agendamento_pkey (3.500 linhas em 23 páginas). Os nós mais custosos em tempo são item_servico (0,553 ms) e agendamento (0,360 ms). As estimativas batem com a realidade (rows estimadas iguais às reais), porque cada COUNT(*) produz exatamente uma linha. Todos os Buffers são shared hit, sem leitura de disco.

Decisão. Para o ramo de agendamento, vale testar um índice secundário mais estreito que agendamento_pkey. Como a tabela agendamento tem várias colunas largas e o PK é apenas o id, um índice sobre id_veiculo (também INT4) tem largura equivalente, mas pode oferecer ordenação útil para outras consultas (Q5) e oferece ao otimizador uma alternativa de menor custo para o Index only Scan puramente para contagem.

CREATE INDEX testado

```
CREATE INDEX idx_agendamento_veiculo ON agendamento(id_veiculo);
```

Plano COM índice (melhor de 3 execuções, 1.772 ms)

```
Append (cost=5.50..476.37 rows=10 width=40) (actual time=0.030..1.636 rows=10 loops=1) Buffers: shared hit=162
read=6 -> Aggregate (cost=5.50..5.51 rows=1 width=40) (actual time=0.028..0.029 rows=1 loops=1) Buffers:
shared hit=3 -> Seq Scan on cliente (cost=0.00..5.00 rows=200 width=0) (actual time=0.009..0.017 rows=200
```

```

loops=1) Buffers: shared hit=3 -> Aggregate (cost=11.25..11.26 rows=1 width=40) (actual time=0.039..0.039
rows=1 loops=1) Buffers: shared hit=5 -> Seq Scan on veiculo (cost=0.00..10.00 rows=500 width=0) (actual
time=0.005..0.024 rows=500 loops=1) Buffers: shared hit=5 -> Aggregate (cost=1.62..1.64 rows=1 width=40)
(actual time=0.008..0.009 rows=1 loops=1) Buffers: shared hit=1 -> Seq Scan on funcionario (cost=0.00..1.50
rows=50 width=0) (actual time=0.004..0.006 rows=50 loops=1) Buffers: shared hit=1 -> Aggregate
(cost=1.25..1.26 rows=1 width=40) (actual time=0.005..0.006 rows=1 loops=1) Buffers: shared hit=1 -> Seq Scan
on tipo_servico (cost=0.00..1.20 rows=20 width=0) (actual time=0.003..0.005 rows=20 loops=1) Buffers: shared
hit=1 -> Aggregate (cost=1.50..1.51 rows=1 width=40) (actual time=0.006..0.006 rows=1 loops=1) Buffers: shared
hit=1 -> Seq Scan on peca (cost=0.00..1.40 rows=40 width=0) (actual time=0.003..0.005 rows=40 loops=1)
Buffers: shared hit=1 -> Aggregate (cost=89.53..89.54 rows=1 width=40) (actual time=0.293..0.294 rows=1
loops=1) Buffers: shared hit=1 read=6 -> Index Only Scan using idx_agendamento_veiculo on agendamento
(cost=0.28..80.78 rows=3500 width=0) (actual time=0.026..0.196 rows=3500 loops=1) Heap Fetches: 0 Buffers:
shared hit=1 read=6 -> Aggregate (cost=146.50..146.51 rows=1 width=40) (actual time=0.457..0.457 rows=1
loops=1) Buffers: shared hit=59 -> Seq Scan on item_servico (cost=0.00..129.00 rows=7000 width=0) (actual
time=0.005..0.253 rows=7000 loops=1) Buffers: shared hit=59 -> Aggregate (cost=101.50..101.51 rows=1 width=40)
(actual time=0.324..0.324 rows=1 loops=1) Buffers: shared hit=39 -> Seq Scan on item_pecas (cost=0.00..89.00
rows=5000 width=0) (actual time=0.003..0.179 rows=5000 loops=1) Buffers: shared hit=39 -> Aggregate
(cost=65.06..65.07 rows=1 width=40) (actual time=0.207..0.207 rows=1 loops=1) Buffers: shared hit=27 -> Seq
Scan on pagamento (cost=0.00..57.45 rows=3045 width=0) (actual time=0.009..0.122 rows=3045 loops=1) Buffers:
shared hit=27 -> Aggregate (cost=52.50..52.51 rows=1 width=40) (actual time=0.262..0.262 rows=1 loops=1)
Buffers: shared hit=25 -> Seq Scan on avaliacao (cost=0.00..47.00 rows=2200 width=0) (actual time=0.124..0.200
rows=2200 loops=1) Buffers: shared hit=25 Planning Time: 0.711 ms Execution Time: 1.772 ms (47 linhas)

```

Comparação quantitativa

Métrica	Sem índice	Com índice	Redução
Tempo total (ms)	1.872	1.772	-5.3%
Tipo de varredura principal (agendamento)	Index Only Scan using agendamento_pkey	Index Only Scan using idx_agendamento_veiculo	troca de índice
Custo estimado do nó agendamento	144,78	80,78	-44%
Buffers do nó agendamento	23 hit	1 hit + 6 read	-70% páginas tocadas
Buffers totais	184 shared hit	162 shared hit + 6 read	leve melhora

Justificativa final. Índice **MANTIDO**. O otimizador efetivamente trocou `agendamento_pkey` por `idx_agendamento_veiculo` no Index Only Scan do ramo de contagem, e o custo estimado do nó caiu de 144,78 para 80,78 (a coluna `id_veiculo` está agendada como FK e seu índice é mais compacto/útil para outras junções). A redução observada é modesta (5%), mas é a única que se materializou efetivamente em escolha do planejador.

Consulta 2: receita total e ticket médio por mês

SQL executado

```

SELECT TO_CHAR(data_conclusao, 'YYYY-MM') AS mes, COUNT(*) AS qtd_agendamentos,
ROUND(SUM(total_geral)::numeric, 2) AS receita_total, ROUND(AVG(total_geral)::numeric, 2) AS ticket_medio FROM
agendamento WHERE status = 'Concluido' AND data_conclusao IS NOT NULL AND data_conclusao >= CURRENT_DATE -
INTERVAL '12 months' GROUP BY TO_CHAR(data_conclusao, 'YYYY-MM') ORDER BY mes DESC;

```

Plano SEM índices (melhor de 3 execuções, 1.343 ms)

```

Sort (cost=238.90..240.58 rows=672 width=104) (actual time=1.324..1.324 rows=13 loops=1) Sort Key:
(to_char(data_conclusao, 'YYYY-MM')::text) DESC Sort Method: quicksort Memory: 25kB Buffers: shared hit=115 ->
HashAggregate (cost=192.22..207.34 rows=672 width=104) (actual time=1.301..1.309 rows=13 loops=1) Group Key:
to_char(data_conclusao, 'YYYY-MM')::text Batches: 1 Memory Usage: 49kB Buffers: shared hit=115 -> Seq Scan on
agendamento (cost=0.00..186.81 rows=722 width=37) (actual time=0.019..1.030 rows=1285 loops=1) Filter:
((data_conclusao IS NOT NULL) AND (status = 'Concluido')::text) AND (data_conclusao >= (CURRENT_DATE - '1
year')::interval)) Rows Removed by Filter: 2215 Buffers: shared hit=115 Planning Time: 0.067 ms Execution
Time: 1.343 ms (14 linhas)

```

Diagnóstico. O nó dominante é o Seq Scan em agendamento (1,030 ms), que percorre as 115 páginas da tabela aplicando filtro composto. O filtro produz 1.285 linhas (e remove 2.215), ou seja, ~37% de seletividade. A estimativa do planejador (722 linhas) ficou abaixo do real (1.285), mas isso não afetou a escolha do plano, porque mesmo a 37% de

seletividade não vale a pena trocar Seq Scan por Index Scan numa tabela de 115 páginas.

Decisão. Hipótese inicial foi criar um índice parcial sobre (status, data_conclusao DESC) WHERE status = 'Concluído' AND data_conclusao IS NOT NULL. A intuição: filtro composto + ordenação descendente poderia ser servido por Index Only Scan.

CREATE INDEX testado

```
CREATE INDEX idx_agendamento_status_data_desc ON agendamento(status, data_conclusao DESC) WHERE status = 'Concluído' AND data_conclusao IS NOT NULL;
```

Plano COM índice (melhor de 3 execuções, 1.318 ms)

```
Sort (cost=238.90..240.58 rows=672 width=104) (actual time=1.300..1.301 rows=13 loops=1) Sort Key:
(to_char(data_conclusao, 'YYYY-MM'::text)) DESC Sort Method: quicksort Memory: 25kB Buffers: shared hit=115 ->
HashAggregate (cost=192.22..207.34 rows=672 width=104) (actual time=1.278..1.285 rows=13 loops=1) Group Key:
to_char(data_conclusao, 'YYYY-MM'::text) Batches: 1 Memory Usage: 49kB Buffers: shared hit=115 -> Seq Scan on
agendamento (cost=0.00..186.81 rows=722 width=37) (actual time=0.017..1.010 rows=1285 loops=1) Filter:
((data_conclusao IS NOT NULL) AND (status = 'Concluído'::text) AND (data_conclusao >= (CURRENT_DATE - '1
year'::interval))) Rows Removed by Filter: 2215 Buffers: shared hit=115 Planning: Buffers: shared hit=2
Planning Time: 0.074 ms Execution Time: 1.318 ms (16 linhas)
```

Comparação quantitativa

Métrica	Sem índice	Com índice	Redução
Tempo total (ms)	1.343	1.318	-1.9%
Tipo de varredura principal	Seq Scan + Filter	Seq Scan + Filter	inalterado
Linhas filtradas (actual)	1.285 (de 3.500)	1.285 (de 3.500)	inalterado
Buffers shared hit	115	115	inalterado

Justificativa final. Índice **DESCARTADO**. O otimizador ignorou o índice parcial criado e manteve o Seq Scan. Razão: 70% dos agendamentos têm status = 'Concluído' (2.450 de 3.500), e o filtro composto adicional só remove a fatia fora dos últimos 12 meses (que é minoria). Com 37% de seletividade efetiva e tabela cabendo em 115 páginas, o custo de varredura sequencial em RAM (cerca de 1 ms) é menor do que ler um índice e fazer lookup heap. A diferença observada (~2%) é ruído de medição.

Consulta 3: top 10 tipos de serviço mais realizados

SQL executado

```
SELECT ts.id_tipo_servico, ts.descricao, SUM(isv.quantidade) AS qtd_execucoes, ROUND(SUM(isv.total)::numeric,
2) AS faturamento FROM item_servico isv JOIN tipo_servico ts ON isv.id_tipo_servico = ts.id_tipo_servico GROUP
BY ts.id_tipo_servico, ts.descricao ORDER BY qtd_execucoes DESC, faturamento DESC LIMIT 10;
```

Plano SEM índices (melhor de 3 execuções, 2.644 ms)

```
Limit (cost=205.95..205.98 rows=10 width=67) (actual time=2.617..2.620 rows=10 loops=1) Buffers: shared hit=60
-> Sort (cost=205.95..206.00 rows=20 width=67) (actual time=2.615..2.618 rows=10 loops=1) Sort Key:
(sum(isv.quantidade)) DESC, (round(sum((isv.total)::numeric), 2)) DESC Sort Method: quicksort Memory: 27kB
Buffers: shared hit=60 -> HashAggregate (cost=205.22..205.52 rows=20 width=67) (actual time=2.596..2.602
rows=20 loops=1) Group Key: ts.id_tipo_servico Batches: 1 Memory Usage: 32kB Buffers: shared hit=60 -> Hash
Join (cost=1.45..152.72 rows=7000 width=36) (actual time=0.027..1.603 rows=7000 loops=1) Hash Cond:
(isv.id_tipo_servico = ts.id_tipo_servico) Buffers: shared hit=60 -> Seq Scan on item_servico isv
(cost=0.00..129.00 rows=7000 width=13) (actual time=0.008..0.311 rows=7000 loops=1) Buffers: shared hit=59 ->
Hash (cost=1.20..1.20 rows=20 width=27) (actual time=0.010..0.011 rows=20 loops=1) Buckets: 1024 Batches: 1
Memory Usage: 10kB Buffers: shared hit=1 -> Seq Scan on tipo_servico ts (cost=0.00..1.20 rows=20 width=27)
(actual time=0.006..0.007 rows=20 loops=1) Buffers: shared hit=1 Planning: Buffers: shared hit=61 Planning
Time: 0.283 ms Execution Time: 2.644 ms (24 linhas)
```

Diagnóstico. O plano é canônico para uma junção fato x dimensão: Hash Join constrói a tabela de hash sobre tipo_servico (20 linhas, 1 página, ~0,01 ms) e percorre item_servico sequencialmente (7.000 linhas, 59 páginas, 1,6

ms). O agregador e o Sort consomem o restante. As estimativas batem (7.000 linhas de fato e 20 de dimensão). Todos os Buffers são shared hit.

Decisão. Hipótese inicial: índice sobre item_servico(id_tipo_servico) poderia permitir um Nested Loop indexado.

CREATE INDEX testado

```
CREATE INDEX idx_item_servico_tipo_servico ON item_servico(id_tipo_servico);
```

Plano COM índice (melhor de 3 execuções, 2.740 ms)

```
Limit (cost=205.95..205.98 rows=10 width=67) (actual time=2.718..2.719 rows=10 loops=1) Buffers: shared hit=60
-> Sort (cost=205.95..206.00 rows=20 width=67) (actual time=2.717..2.718 rows=10 loops=1) Sort Key:
(sum(isv.quantidade)) DESC, (round(sum((isv.total)::numeric), 2)) DESC Sort Method: quicksort Memory: 27kB
Buffers: shared hit=60 -> HashAggregate (cost=205.22..205.52 rows=20 width=67) (actual time=2.700..2.706
rows=20 loops=1) Group Key: ts.id_tipo_servico Batches: 1 Memory Usage: 32kB Buffers: shared hit=60 -> Hash
Join (cost=1.45..152.72 rows=7000 width=36) (actual time=0.022..1.662 rows=7000 loops=1) Hash Cond:
(isv.id_tipo_servico = ts.id_tipo_servico) Buffers: shared hit=60 -> Seq Scan on item_servico isv
(cost=0.00..129.00 rows=7000 width=13) (actual time=0.006..0.318 rows=7000 loops=1) Buffers: shared hit=59 ->
Hash (cost=1.20..1.20 rows=20 width=27) (actual time=0.009..0.009 rows=20 loops=1) Buckets: 1024 Batches: 1
Memory Usage: 10kB Buffers: shared hit=1 -> Seq Scan on tipo_servico ts (cost=0.00..1.20 rows=20 width=27)
(actual time=0.005..0.007 rows=20 loops=1) Buffers: shared hit=1 Planning: Buffers: shared hit=6 Planning
Time: 0.118 ms Execution Time: 2.740 ms (24 linhas)
```

Comparação quantitativa

Métrica	Sem índice	Com índice	Redução
Tempo total (ms)	2.644	2.740	+3.6% (ruído)
Tipo de junção	Hash Join	Hash Join	inalterado
Linhas processadas no join	7.000	7.000	inalterado
Buffers shared hit	60	60	inalterado

Justificativa final. Índice **DESCARTADO**. O otimizador continuou escolhendo Hash Join, mesmo com o índice disponível. O motivo é matemático: construir o hash sobre tipo_servico (20 linhas) custa O(20) e a varredura de item_servico é O(7.000). Um Nested Loop indexado faria 7.000 lookups em B-tree (cada lookup ~log2(20) ≈ 5 níveis, custo agregado bem maior). O custo estimado idêntico nos dois planos (152,72) confirma que o planejador avaliou ambas as alternativas e preferiu o Hash Join quando o índice existe (não há mudança de plano).

Consulta 4: ranking de funcionários por faturamento

SQL executado

```
SELECT f.id_funcionario, f.nome, COUNT(DISTINCT isv.id_agendamento) AS qtd_os, ROUND(SUM(isv.total)::numeric,
2) AS faturamento FROM funcionario f JOIN item_servico isv ON isv.id_funcionario = f.id_funcionario JOIN
agendamento ag ON ag.id_agendamento = isv.id_agendamento WHERE ag.status = 'Concluido' GROUP BY
f.id_funcionario, f.nome ORDER BY faturamento DESC;
```

Plano SEM índices (melhor de 3 execuções, 5.238 ms)

```
Sort (cost=704.35..704.47 rows=50 width=58) (actual time=5.168..5.171 rows=50 loops=1) Sort Key:
(round(sum((isv.total)::numeric), 2)) DESC Sort Method: quicksort Memory: 28kB Buffers: shared hit=175 ->
GroupAggregate (cost=653.18..702.93 rows=50 width=58) (actual time=4.535..5.154 rows=50 loops=1) Group Key:
f.id_funcionario Buffers: shared hit=175 -> Sort (cost=653.18..665.43 rows=4900 width=27) (actual
time=4.512..4.649 rows=4900 loops=1) Sort Key: f.id_funcionario, isv.id_agendamento Sort Method: quicksort
Memory: 422kB Buffers: shared hit=175 -> Hash Join (cost=191.50..352.85 rows=4900 width=27) (actual
time=0.561..2.583 rows=4900 loops=1) Hash Cond: (isv.id_funcionario = f.id_funcionario) Buffers: shared
hit=175 -> Hash Join (cost=189.38..336.77 rows=4900 width=13) (actual time=0.540..1.874 rows=4900 loops=1)
Hash Cond: (isv.id_agendamento = ag.id_agendamento) Buffers: shared hit=174 -> Seq Scan on item_servico isv
(cost=0.00..129.00 rows=7000 width=13) (actual time=0.004..0.327 rows=7000 loops=1) Buffers: shared hit=59 ->
Hash (cost=158.75..158.75 rows=2450 width=4) (actual time=0.534..0.534 rows=2450 loops=1) Buckets: 4096
Batches: 1 Memory Usage: 119kB Buffers: shared hit=115 -> Seq Scan on agendamento ag (cost=0.00..158.75
rows=2450 width=4) (actual time=0.011..0.335 rows=2450 loops=1) Filter: (status = 'Concluido'::text) Rows
```

Removed by Filter: 1050 Buffers: shared hit=115 -> Hash (cost=1.50..1.50 rows=50 width=18) (actual time=0.017..0.017 rows=50 loops=1) Buckets: 1024 Batches: 1 Memory Usage: 11kB Buffers: shared hit=1 -> Seq Scan on funcionario f (cost=0.00..1.50 rows=50 width=18) (actual time=0.008..0.011 rows=50 loops=1) Buffers: shared hit=1 Planning: Buffers: shared hit=12 Planning Time: 0.171 ms Execution Time: 5.238 ms (35 linhas)

Diagnóstico. Plano composto: duplo Hash Join (item_servico ■ agendamento ■ funcionario), seguido de Sort para suportar GroupAggregate com COUNT(DISTINCT ...), e Sort final para ORDER BY. O nó mais caro é o Sort interno (4.900 linhas, 422 kB de memória, 4,6 ms — domina o tempo total). Os Hash Joins consomem ~2,6 ms. As estimativas estão precisas (4.900 reais vs 4.900 esperadas). Buffers integralmente em cache.

Decisão. Foram testados dois índices: um sobre item_servico(id_funcionario) (motivado pelo Hash Cond) e um parcial sobre agendamento(status) WHERE status='Concluído' (motivado pelo filtro de status que remove 1.050 linhas).

CREATE INDEX testado

```
CREATE INDEX idx_item_servico_funcionario ON item_servico(id_funcionario); CREATE INDEX
idx_agendamento_status_concluido ON agendamento(status) WHERE status = 'Concluído';
```

Plano COM índice (melhor de 3 execuções, 5.277 ms)

Sort (cost=704.35..704.47 rows=50 width=58) (actual time=5.204..5.206 rows=50 loops=1) Sort Key: (round(sum((isv.total)::numeric), 2)) DESC Sort Method: quicksort Memory: 28kB Buffers: shared hit=175 -> GroupAggregate (cost=653.18..702.93 rows=50 width=58) (actual time=4.431..5.189 rows=50 loops=1) Group Key: f.id_funcionario Buffers: shared hit=175 -> Sort (cost=653.18..665.43 rows=4900 width=27) (actual time=4.410..4.553 rows=4900 loops=1) Sort Key: f.id_funcionario, isv.id_agendamento Sort Method: quicksort Memory: 422kB Buffers: shared hit=175 -> Hash Join (cost=191.50..352.85 rows=4900 width=27) (actual time=0.580..2.593 rows=4900 loops=1) Hash Cond: (isv.id_funcionario = f.id_funcionario) Buffers: shared hit=175 -> Hash Join (cost=189.38..336.77 rows=4900 width=13) (actual time=0.556..1.872 rows=4900 loops=1) Hash Cond: (isv.id_agendamento = ag.id_agendamento) Buffers: shared hit=174 -> Seq Scan on item_servico isv (cost=0.00..129.00 rows=7000 width=13) (actual time=0.003..0.302 rows=7000 loops=1) Buffers: shared hit=59 -> Hash (cost=158.75..158.75 rows=2450 width=4) (actual time=0.541..0.541 rows=2450 loops=1) Buckets: 4096 Batches: 1 Memory Usage: 119kB Buffers: shared hit=115 -> Seq Scan on agendamento ag (cost=0.00..158.75 rows=2450 width=4) (actual time=0.011..0.329 rows=2450 loops=1) Filter: (status = 'Concluído'::text) Rows Removed by Filter: 1050 Buffers: shared hit=115 -> Hash (cost=1.50..1.50 rows=50 width=18) (actual time=0.019..0.020 rows=50 loops=1) Buckets: 1024 Batches: 1 Memory Usage: 11kB Buffers: shared hit=1 -> Seq Scan on funcionario f (cost=0.00..1.50 rows=50 width=18) (actual time=0.008..0.011 rows=50 loops=1) Buffers: shared hit=1 Planning: Buffers: shared hit=14 Planning Time: 0.210 ms Execution Time: 5.277 ms (35 linhas)

Comparação quantitativa

Métrica	Sem índice	Com índice	Redução
Tempo total (ms)	5.238	5.277	+0.7% (equivalente)
Estrutura do plano	duplo Hash Join + Sort + GroupAggregate	idêntica	inalterado
Linhas no join intermediário	4.900	4.900	inalterado
Buffers shared hit	175	175	inalterado

Justificativa final. Ambos os índices **DESCARTADOS**. O planejador não os utilizou. A lógica é a mesma de Q3: construir hash sobre 50 funcionários custa quase nada; o filtro status='Concluído' mantém 70% das linhas, seletividade insuficiente para o índice parcial vencer o Seq Scan em 115 páginas. O custo dominante (Sort de 4.900 linhas) não pode ser eliminado por índices porque a ordenação é por uma expressão composta após a junção.

Consulta 5: top 20 clientes por gasto acumulado

SQL executado

```
SELECT c.id_cliente, c.nome, CASE WHEN c.cpf IS NOT NULL THEN 'PF' ELSE 'PJ' END AS tipo_cliente,
ROUND(SUM(p.valor)::numeric, 2) AS gasto_total FROM cliente c JOIN veiculo v ON v.id_cliente = c.id_cliente
JOIN agendamento ag ON ag.id_veiculo = v.id_veiculo JOIN pagamento p ON p.id_agendamento = ag.id_agendamento
GROUP BY c.id_cliente, c.nome, tipo_cliente ORDER BY gasto_total DESC LIMIT 20;
```


Plano SEM índices (melhor de 3 execuções, 2.849 ms)

```
Limit (cost=330.33..330.38 rows=20 width=81) (actual time=2.800..2.802 rows=20 loops=1) Buffers: shared hit=150 -> Sort (cost=330.33..330.83 rows=200 width=81) (actual time=2.799..2.801 rows=20 loops=1) Sort Key: (round(sum((p.valor)::numeric), 2)) DESC Sort Method: top-N heapsort Memory: 27kB Buffers: shared hit=150 -> HashAggregate (cost=322.01..325.01 rows=200 width=81) (actual time=2.738..2.766 rows=140 loops=1) Group Key: c.id_cliente, CASE WHEN (c.cpf IS NOT NULL) THEN 'PF'::text ELSE 'PJ'::text END Batches: 1 Memory Usage: 96kB Buffers: shared hit=150 -> Hash Join (cost=217.50..299.17 rows=3045 width=54) (actual time=0.725..2.162 rows=3045 loops=1) Hash Cond: (v.id_cliente = c.id_cliente) Buffers: shared hit=150 -> Hash Join (cost=210.00..283.51 rows=3045 width=9) (actual time=0.672..1.710 rows=3045 loops=1) Hash Cond: (ag.id_veiculo = v.id_veiculo) Buffers: shared hit=147 -> Hash Join (cost=193.75..259.20 rows=3045 width=9) (actual time=0.587..1.272 rows=3045 loops=1) Hash Cond: (p.id_agendamento = ag.id_agendamento) Buffers: shared hit=142 -> Seq Scan on pagamento p (cost=0.00..57.45 rows=3045 width=9) (actual time=0.003..0.123 rows=3045 loops=1) Buffers: shared hit=27 -> Hash (cost=150.00..150.00 rows=3500 width=8) (actual time=0.582..0.582 rows=3500 loops=1) Buckets: 4096 Batches: 1 Memory Usage: 169kB Buffers: shared hit=115 -> Seq Scan on agendamento ag (cost=0.00..150.00 rows=3500 width=8) (actual time=0.012..0.263 rows=3500 loops=1) Buffers: shared hit=115 -> Hash (cost=10.00..10.00 rows=500 width=8) (actual time=0.083..0.083 rows=500 loops=1) Buckets: 1024 Batches: 1 Memory Usage: 28kB Buffers: shared hit=5 -> Seq Scan on veiculo v (cost=0.00..10.00 rows=500 width=8) (actual time=0.005..0.041 rows=500 loops=1) Buffers: shared hit=5 -> Hash (cost=5.00..5.00 rows=200 width=29) (actual time=0.050..0.050 rows=200 loops=1) Buckets: 1024 Batches: 1 Memory Usage: 20kB Buffers: shared hit=3 -> Seq Scan on cliente c (cost=0.00..5.00 rows=200 width=29) (actual time=0.009..0.026 rows=200 loops=1) Buffers: shared hit=3 Planning: Buffers: shared hit=18 Planning Time: 0.299 ms Execution Time: 2.849 ms (40 linhas)
```

Diagnóstico. Triple Hash Join em cascata (pagamento ■ agendamento ■ veiculo ■ cliente). Tabelas pequenas (cliente 200, veículo 500) entram no lado de construção do hash; agendamento (3.500) é construído como hash intermediário. O fluxo principal é dirigido pelo Seq Scan em pagamento (3.045 linhas). Tempo dominado pelo HashAggregate final (2,77 ms). Buffers todos em cache.

Decisão. Foram testados dois índices em colunas de junção: veiculo(id_cliente) e pagamento(id_agendamento).

CREATE INDEX testado

```
CREATE INDEX idx_veiculo_cliente ON veiculo(id_cliente); CREATE INDEX idx_pagamento_agendamento ON pagamento(id_agendamento);
```

Plano COM índice (melhor de 3 execuções, 3.042 ms)

```
Limit (cost=330.33..330.38 rows=20 width=81) (actual time=2.991..2.993 rows=20 loops=1) Buffers: shared hit=150 -> Sort (cost=330.33..330.83 rows=200 width=81) (actual time=2.990..2.992 rows=20 loops=1) Sort Key: (round(sum((p.valor)::numeric), 2)) DESC Sort Method: top-N heapsort Memory: 27kB Buffers: shared hit=150 -> HashAggregate (cost=322.01..325.01 rows=200 width=81) (actual time=2.926..2.957 rows=140 loops=1) Group Key: c.id_cliente, CASE WHEN (c.cpf IS NOT NULL) THEN 'PF'::text ELSE 'PJ'::text END Batches: 1 Memory Usage: 96kB Buffers: shared hit=150 -> Hash Join (cost=217.50..299.17 rows=3045 width=54) (actual time=0.705..2.342 rows=3045 loops=1) Hash Cond: (v.id_cliente = c.id_cliente) Buffers: shared hit=150 -> Hash Join (cost=210.00..283.51 rows=3045 width=9) (actual time=0.655..1.746 rows=3045 loops=1) Hash Cond: (ag.id_veiculo = v.id_veiculo) Buffers: shared hit=147 -> Hash Join (cost=193.75..259.20 rows=3045 width=9) (actual time=0.569..1.261 rows=3045 loops=1) Hash Cond: (p.id_agendamento = ag.id_agendamento) Buffers: shared hit=142 -> Seq Scan on pagamento p (cost=0.00..57.45 rows=3045 width=9) (actual time=0.003..0.149 rows=3045 loops=1) Buffers: shared hit=27 -> Hash (cost=150.00..150.00 rows=3500 width=8) (actual time=0.563..0.563 rows=3500 loops=1) Buckets: 4096 Batches: 1 Memory Usage: 169kB Buffers: shared hit=115 -> Seq Scan on agendamento ag (cost=0.00..150.00 rows=3500 width=8) (actual time=0.009..0.253 rows=3500 loops=1) Buffers: shared hit=115 -> Hash (cost=10.00..10.00 rows=500 width=8) (actual time=0.082..0.083 rows=500 loops=1) Buckets: 1024 Batches: 1 Memory Usage: 28kB Buffers: shared hit=5 -> Seq Scan on veiculo v (cost=0.00..10.00 rows=500 width=8) (actual time=0.005..0.040 rows=500 loops=1) Buffers: shared hit=5 -> Hash (cost=5.00..5.00 rows=200 width=29) (actual time=0.048..0.048 rows=200 loops=1) Buckets: 1024 Batches: 1 Memory Usage: 20kB Buffers: shared hit=3 -> Seq Scan on cliente c (cost=0.00..5.00 rows=200 width=29) (actual time=0.009..0.025 rows=200 loops=1) Buffers: shared hit=3 Planning: Buffers: shared hit=38 Planning Time: 0.418 ms Execution Time: 3.042 ms (40 linhas)
```

Comparação quantitativa

Métrica	Sem índice	Com índice	Redução
Tempo total (ms)	2.849	3.042	+6.8% (ruído)
Estrutura do plano	triple Hash Join + HashAggregate	idêntica	inalterado
Linhas no fluxo principal	3.045	3.045	inalterado
Buffers shared hit	150	150	inalterado

Justificativa final. Ambos os índices **DESCARTADOS**. O planejador não substituiu nenhum Hash Join por Nested Loop indexado. Construir o hash de `cliente` (200 linhas) e de `veiculo` (500 linhas) é trivialmente barato, e percorrer `pagamento/agendamento` sequencialmente é mais rápido do que sondar B-trees milhares de vezes. A diferença observada (~7%) é ruído normal de medição em escalas de poucos milissegundos.

Consulta 6: distribuição percentual das formas de pagamento

SQL executado

```
SELECT forma_pagamento, COUNT(*) AS qtd_transacoes, ROUND(SUM(valor)::numeric, 2) AS valor_total, ROUND(
(SUM(valor) * 100.0 / (SELECT SUM(valor) FROM pagamento))::numeric, 2 ) AS perc_valor FROM pagamento GROUP BY
forma_pagamento ORDER BY valor_total DESC;
```

Plano SEM índices (melhor de 3 execuções, 1.216 ms)

```
Sort (cost=145.53..145.55 rows=5 width=85) (actual time=1.198..1.198 rows=5 loops=1) Sort Key:
(round(sum((pagamento.valor)::numeric), 2)) DESC Sort Method: quicksort Memory: 25kB Buffers: shared hit=54
InitPlan 1 -> Aggregate (cost=65.06..65.08 rows=1 width=32) (actual time=0.412..0.412 rows=1 loops=1) Buffers:
shared hit=27 -> Seq Scan on pagamento pagamento_1 (cost=0.00..57.45 rows=3045 width=5) (actual
time=0.003..0.131 rows=3045 loops=1) Buffers: shared hit=27 -> HashAggregate (cost=80.29..80.40 rows=5
width=85) (actual time=1.191..1.195 rows=5 loops=1) Group Key: pagamento.forma_pagamento Batches: 1 Memory
Usage: 24kB Buffers: shared hit=54 -> Seq Scan on pagamento (cost=0.00..57.45 rows=3045 width=18) (actual
time=0.005..0.136 rows=3045 loops=1) Buffers: shared hit=27 Planning Time: 0.057 ms Execution Time: 1.216 ms
(17 linhas)
```

Diagnóstico. Dois Seq Scan sobre pagamento (3.045 linhas, 27 páginas cada): um para o InitPlan (SUM total para o percentual) e outro para o HashAggregate principal. Cada varredura toma ~0,13 ms; o agregador domina o tempo (~1,2 ms no total). Não há filtro nem junção: a consulta lê todas as 3.045 linhas obrigatoriamente para produzir as somas por `forma_pagamento`.

Decisão. Nenhum índice ajudaria. Um índice em `forma_pagamento` não evita a leitura integral da tabela (GROUP BY precisa de todas as linhas para somar); um Index Scan ordenado por `forma_pagamento` poderia substituir o HashAggregate por GroupAggregate agrupado, mas como há apenas 5 valores distintos numa tabela de 3.045 linhas, o ganho seria nulo ou negativo.

CREATE INDEX testado. Nenhum índice foi proposto para esta consulta.

Justificativa final. SEM ÍNDICE PROPOSTO. Toda a tabela é necessariamente lida; o tempo é dominado pela agregação e pelo cálculo do percentual via `InitPlan`. A consulta já está perto do ótimo teórico.

Consulta 7: peças com estoque abaixo do mínimo

SQL executado

```
SELECT id_pecas, nome, quantidade_estoque, quantidade_minima, (quantidade_minima - quantidade_estoque) AS
deficit, fornecedor FROM pecas WHERE quantidade_estoque < quantidade_minima ORDER BY deficit DESC;
```

Plano SEM índices (melhor de 3 execuções, 0.019 ms)

```
Sort (cost=1.77..1.81 rows=13 width=41) (actual time=0.013..0.013 rows=10 loops=1) Sort Key:
((quantidade_minima - quantidade_estoque)) DESC Sort Method: quicksort Memory: 25kB Buffers: shared hit=1 ->
Seq Scan on pecas (cost=0.00..1.53 rows=13 width=41) (actual time=0.006..0.009 rows=10 loops=1) Filter:
(quantidade_estoque < quantidade_minima) Rows Removed by Filter: 30 Buffers: shared hit=1 Planning Time: 0.025
ms Execution Function 0.019 ms (10 linhas)
```

Diagnóstico. Seq Scan em pecas (40 linhas, 1 página, 0,009 ms) com filtro de comparação entre duas colunas (`quantidade_estoque < quantidade_minima`). Tempo total 19 microssegundos. A tabela inteira cabe em uma única página de 8 KB.

Decisão. Índices em filtros que comparam duas colunas exigiriam um índice de expressão (`((quantidade_minima - quantidade_estoque))`) e seriam tecnicamente possíveis, mas inúteis: ler um índice tem custo mínimo equivalente, e a tabela de 40 linhas é varrida em um único acesso a página. Qualquer índice degradaria o desempenho de INSERT/UPDATE sem benefício mensurável.

CREATE INDEX testado. Nenhum índice foi proposto para esta consulta.

Justificativa final. SEM ÍNDICE PROPOSTO. Tabela trivialmente pequena; o tempo total de 19 µs já está abaixo de qualquer ganho possível.

Consulta 8: nota média por funcionário

SQL executado

```
SELECT f.id_funcionario, f.nome, COUNT(DISTINCT av.id_avaliacao) AS qtd_avaliacoes,
ROUND(AVG(av.nota)::numeric, 2) AS nota_media FROM funcionario f JOIN item_servico isv ON isv.id_funcionario =
f.id_funcionario JOIN avaliacao av ON av.id_agendamento = isv.id_agendamento GROUP BY f.id_funcionario, f.nome
HAVING COUNT(DISTINCT av.id_avaliacao) >= 5 ORDER BY nota_media DESC;
```

Plano SEM índices (melhor de 3 execuções, 4.184 ms)

```
Sort (cost=547.96..548.01 rows=17 width=58) (actual time=4.066..4.068 rows=50 loops=1) Sort Key:
(round(avg(av.nota), 2)) DESC Sort Method: quicksort Memory: 27kB Buffers: shared hit=85 -> GroupAggregate
(cost=502.82..547.62 rows=17 width=58) (actual time=3.580..4.053 rows=50 loops=1) Group Key: f.id_funcionario
Filter: (count(DISTINCT av.id_avaliacao) >= 5) Buffers: shared hit=85 -> Sort (cost=502.82..513.82 rows=4400
width=26) (actual time=3.562..3.688 rows=4400 loops=1) Sort Key: f.id_funcionario, av.id_avaliacao Sort
Method: quicksort Memory: 399kB Buffers: shared hit=85 -> Hash Join (cost=76.62..236.55 rows=4400 width=26)
(actual time=0.417..2.077 rows=4400 loops=1) Hash Cond: (isv.id_funcionario = f.id_funcionario) Buffers:
shared hit=85 -> Hash Join (cost=74.50..221.90 rows=4400 width=12) (actual time=0.396..1.487 rows=4400
loops=1) Hash Cond: (isv.id_agendamento = av.id_agendamento) Buffers: shared hit=84 -> Seq Scan on
item_servico isv (cost=0.00..129.00 rows=7000 width=8) (actual time=0.004..0.281 rows=7000 loops=1) Buffers:
shared hit=59 -> Hash (cost=47.00..47.00 rows=2200 width=12) (actual time=0.388..0.388 rows=2200 loops=1)
Buckets: 4096 Batches: 1 Memory Usage: 127kB Buffers: shared hit=25 -> Seq Scan on avaliacao av
(cost=0.00..47.00 rows=2200 width=12) (actual time=0.004..0.170 rows=2200 loops=1) Buffers: shared hit=25 ->
Hash (cost=1.50..1.50 rows=50 width=18) (actual time=0.018..0.018 rows=50 loops=1) Buckets: 1024 Batches: 1
Memory Usage: 11kB Buffers: shared hit=1 -> Seq Scan on funcionario f (cost=0.00..1.50 rows=50 width=18)
(actual time=0.008..0.011 rows=50 loops=1) Buffers: shared hit=1 Planning: Buffers: shared hit=26 Planning
Time: 0.214 ms Execution Time: 4.184 ms (34 linhas)
```

Diagnóstico. Estrutura semelhante a Q4: duplo Hash Join (item_servico ■ avaliacao ■ funcionario), Sort de 4.400 linhas (399 kB, 3,69 ms — dominante), GroupAggregate com filtro HAVING. Estimativas razoavelmente próximas (4.400 reais vs. 4.400 esperadas; agregação subestimada em 17 vs. 50 grupos, mas sem impacto prático). Buffers todos em cache.

Decisão. Foi testado um índice sobre item_servico(id_funcionario) específico para Q8 (variante distinct). Em prática, é o mesmo índice já testado para Q4.

CREATE INDEX testado

```
CREATE INDEX idx_item_servico_funcionario_distinct ON item_servico(id_funcionario);
```

Plano COM índice (melhor de 3 execuções, 4.637 ms)

```
Sort (cost=547.96..548.01 rows=17 width=58) (actual time=4.638..4.640 rows=50 loops=1) Sort Key:
(round(avg(av.nota), 2)) DESC Sort Method: quicksort Memory: 27kB Buffers: shared hit=85 -> GroupAggregate
(cost=502.82..547.62 rows=17 width=58) (actual time=3.977..4.624 rows=50 loops=1) Group Key: f.id_funcionario
Filter: (count(DISTINCT av.id_avaliacao) >= 5) Buffers: shared hit=85 -> Sort (cost=502.82..513.82 rows=4400
width=26) (actual time=3.956..4.095 rows=4400 loops=1) Sort Key: f.id_funcionario, av.id_avaliacao Sort
Method: quicksort Memory: 399kB Buffers: shared hit=85 -> Hash Join (cost=76.62..236.55 rows=4400 width=26)
(actual time=0.395..2.216 rows=4400 loops=1) Hash Cond: (isv.id_funcionario = f.id_funcionario) Buffers:
shared hit=85 -> Hash Join (cost=74.50..221.90 rows=4400 width=12) (actual time=0.375..1.596 rows=4400
loops=1) Hash Cond: (isv.id_agendamento = av.id_agendamento) Buffers: shared hit=84 -> Seq Scan on
item_servico isv (cost=0.00..129.00 rows=7000 width=8) (actual time=0.003..0.315 rows=7000 loops=1) Buffers:
shared hit=59 -> Hash (cost=47.00..47.00 rows=2200 width=12) (actual time=0.369..0.369 rows=2200 loops=1)
Buckets: 4096 Batches: 1 Memory Usage: 127kB Buffers: shared hit=25 -> Seq Scan on avaliacao av
(cost=0.00..47.00 rows=2200 width=12) (actual time=0.003..0.165 rows=2200 loops=1) Buffers: shared hit=25 ->
Hash (cost=1.50..1.50 rows=50 width=18) (actual time=0.017..0.017 rows=50 loops=1) Buckets: 1024 Batches: 1
Memory Usage: 11kB Buffers: shared hit=1 -> Seq Scan on funcionario f (cost=0.00..1.50 rows=50 width=18)
(actual time=0.008..0.011 rows=50 loops=1) Buffers: shared hit=1 Planning: Buffers: shared hit=12 Planning
Time: 0.198 ms Execution Time: 4.637 ms (34 linhas)
```

Comparação quantitativa

Métrica	Sem índice	Com índice	Redução
Tempo total (ms)	4.184	4.637	+10.8% (ruído)
Estrutura do plano	duplo Hash Join + Sort + GroupAggregate	idêntica	inalterado
Linhas no Sort intermediário	4.400	4.400	inalterado
Buffers shared hit	85	85	inalterado

Justificativa final. Índice **DESCARTADO**. Plano idêntico, sem mudança de método de junção. Além disso, este índice seria redundante com o já testado em Q4 (`idx_item_servico_funcionario`). A variação de +10,8% é ruído natural quando se trabalha em faixa de poucos milissegundos com cache compartilhado.

3. Síntese dos índices criados

Após o ciclo de validação, apenas um índice foi mantido em `db/05_indices.sql`:

Índice	Tabela	Coluna(s)	Consulta motivadora	Redução observada
<code>idx_agendamento_veiculo</code>	<code>agendamento</code>	<code>id_veiculo</code>	Q1	~5%

Esse índice é, simultaneamente, útil como índice de FK (`id_veiculo` referencia `veiculo`) e tem comprovação empírica de uso pelo planejador para `Index Only Scan` no ramo de contagem de agendamentos em Q1.

4. Índices testados e descartados

Índice testado	Tabela	Coluna(s)	Consulta-alvo	Motivo do descarte
<code>idx_agendamento_status_agendamento</code>	<code>agendamento</code>	<code>status</code> , <code>data_conclusao</code> DESC (partial)	Q2	Otimizador manteve Seq Scan; selectividade baixa (70% Concluído)
<code>idx_item_servico_tipo_servico</code>	<code>item_servico</code>	<code>id_tipo_servico</code>	Q3	Hash Join continua ótimo (20 linhas no lado pequeno)
<code>idx_item_servico_funcionario_item_servico</code>	<code>item_servico</code>	<code>id_funcionario</code>	Q4, Q8	Hash Join continua ótimo (50 funcionários)
<code>idx_agendamento_status_agendamento</code>	<code>agendamento</code>	<code>status</code> (partial WHERE <code>status='Concluído'</code>)	Q4	Partial não selecionado; baixa selectividade
<code>idx_veiculo_cliente</code>	<code>veiculo</code>	<code>id_cliente</code>	Q5	Hash Join continua ótimo (200 clientes)
<code>idx_pagamento_agendamento_pagamento</code>	<code>pagamento</code>	<code>id_agendamento</code>	Q5	Hash Join continua ótimo
<code>idx_item_servico_funcionario_item_servico</code>	<code>item_servico</code>	<code>id_funcionario</code>	Q8	Redundante com <code>idx_item_servico_funcionario</code>

5. Discussão e limitações

O comportamento observado neste estudo é coerente com a teoria do otimizador `cost-based` do PostgreSQL. O volume total carregado equivale a aproximadamente 22 mil registros, cuja heap ocupa cerca de 250 KB distribuídos em poucas centenas de páginas de 8 KB. Todos os planos colhidos exibem `Buffers: shared hit` (a quase totalidade dos dados está em `shared_buffers`), com leituras físicas (`read=N`) ocorrendo apenas marginalmente após a criação do novo índice em Q1. Sem I/O de disco a evitar, índices perdem sua principal vantagem econômica: a redução de páginas lidas.

A segunda razão pela qual a maioria dos índices testados foi rejeitada pelo planejador é estrutural: as junções recorrentes deste schema envolvem tabelas-dimensão muito pequenas (`tipo_servico` com 20, `funcionario` com 50, `cliente` com 200, `veiculo` com 500). Construir uma tabela de hash sobre essas dimensões custa entre 10 e 100 microssegundos, enquanto sondar repetidamente uma B-tree para 4–7 mil linhas do lado de fato custaria várias centenas de microssegundos adicionais. O `Hash Join` é objetivamente melhor neste regime, e o otimizador comprovadamente o reconhece.

Em produção real, com centenas de milhares de ordens de serviço acumuladas ao longo de anos, a balança tende a inverter. Índices em FKs como `item_servico(id_funcionario)`, `pagamento(id_agendamento)` e `veiculo(id_cliente)`, e em colunas de filtro como `agendamento(status, data_conclusao)`, provavelmente passariam a ser selecionados pelo otimizador, especialmente para junções dirigidas por filtros restritivos. A decisão neste relatório de não mantê-los preventivamente respeita o critério do enunciado: manter apenas aquilo que demonstrou ganho empírico verificável neste dataset.

Cabe um esclarecimento sobre os campos derivados `total_servicos` e `total_pecas` em `agendamento`. A versão final do DDL utiliza triggers `BEFORE INSERT/UPDATE/DELETE` em `item_servico` e `item_pecas` para manter esses totais consistentes. A alternativa pretendida originalmente — colunas `GENERATED ALWAYS AS (...) STORED` — não é viável no PostgreSQL porque expressões de coluna gerada armazenada não aceitam subconsultas referenciando outras tabelas (`SUM(item_servico.total) WHERE id_agendamento = ...` é cross-tabela). A regra do enunciado de "valores mantidos automaticamente pelo banco, sem intervenção da aplicação" é, portanto, cumprida pelos triggers definidos no DDL.

6. Conclusão

A análise sistemática dos planos `EXPLAIN ANALYZE` confirmou que, para o volume de dados especificado pelo enunciado, o planejador do PostgreSQL toma decisões corretas e dificilmente melhoráveis: `Seq Scan` quando a tabela é pequena ou o filtro pouco seletivo, `Hash Join` para combinar tabelas pequenas com tabelas de fato. Dos oito índices candidatos avaliados, apenas `idx_agendamento_veiculo` produziu evidência clara e reproduzível de uso pelo otimizador, com ganho de cerca de 5% no tempo total de Q1, e foi consequentemente mantido em `db/05_indices.sql`.

O resultado também é didático: ilustra que índices não são bem-vindos por padrão e que a criação preventiva pode até prejudicar o sistema (sobrecarga em `INSERT/UPDATE/DELETE`, fragmentação do `shared_buffers`). O critério empírico adotado — manter apenas o que comprovou benefício no plano selecionado — é a postura tecnicamente correta para esse cenário.